

Fuzzy Brain

What is FuzzyBrain?

FuzzyBrain is a lightweight behaviour system for Unity. It replaces hand-written `if/else` chains or custom state machines with a prioritised list of **Acts**, each guarded by one or more **Conditions**. Every tick the system evaluates all acts against the current world state and fires the most specific match — the act with the most conditions that all pass simultaneously.

The system is built around three principles: **Component-based** — `Actor` is a drop-in `MonoBehaviour` and behaviour logic lives in small focused components called via `ActContext`. **Separation of data and behaviour** — Conditions are pure data assets; Acts are subclassed `ScriptableObjects` with behaviour implemented in code. **Cheap at scale** — `FuzzyBrainManager` batches evaluations into staggered time buckets across frames.

Core Concepts

`Condition` — `ScriptableObject` that answers a yes/no question about one component on the actor. `Act` — Abstract `ScriptableObject` subclass. Each subclass implements one behaviour in `PerformAct`. `ScriptableActList` — `ScriptableObject` holding an ordered list of Acts for one actor. `Actor` — `MonoBehaviour` host. Manages state, component cache, idle tracking, and evaluation loop. `ActContext` — Zero-allocation per-tick struct carrying the component cache through evaluation. `FuzzyBrainManager` — Singleton driving all Actor evaluations. Auto-created at runtime if absent. `ActHistory` — Optional per-actor ring buffer of recently completed acts. Enables combo sequencing.

Evaluation Pipeline

```
FuzzyBrainManager.Tick()
└─ Actor.ActorUpdate()
    └─ Build ActContext
    └─ [if locked] Poll IsComplete / ActClock timeout
        └─ Complete → unlock, record in ActHistory, proceed
        └─ Not complete → return early (stay locked)
    └─ For each Act in ScriptableActList (specificity-first):
        └─ Act.CheckConditions(ctx)
            └─ Condition → ActContext.Evaluate()
                └─ Cache hit → return cached result
                └─ Cache miss → Condition.Verify() → cache
        └─ [all pass] → first match wins
            └─ Act.OnStart(ctx) ← one-shot setup
            └─ Act.PerformAct(ctx) ← implemented per subclass
            └─ [IsComplete = false] → lock actor, start ActClock
```

Specificity-First Priority & Act Lifecycle

Acts are automatically sorted **descending by condition count** when the act list loads. More-specific acts are always checked before less-specific ones. An act with zero conditions is a guaranteed fallback — it always fires when nothing else matches.

```
Act - 3 conditions    ← checked first
Act - 1 condition
Act - 0 conditions    ← fallback, always fires
```

OnStart — First tick this act is selected. One-shot. Use for animations, sounds, or setup.

PerformAct — Same tick as **OnStart**. Required override. May be empty if **OnStart** does all work. **IsComplete** — Every tick while locked. Return **true** to unlock. Default always returns **true**. **maxClockTime** — Every tick while locked. Forces unlock when exceeded. **0** = no timeout.

Minimal Setup

1. Right-click the Hierarchy → **FuzzyBrain > Actor**. Creates a GameObject with **Actor** pre-attached and adds a **FuzzyBrainManager** to the scene if one does not exist.
2. Create a **ScriptableActList** asset and assign it to the Actor's **Acts** field.
3. Open **Act Wizard > Generate Script**, scaffold a new **Act** subclass, and implement **PerformAct**.
4. Open **Act Wizard > Create Asset**, create an asset instance of your subclass.
5. Assign Condition assets to the Act, then add the Act to the Act List.

For combo support, add an **ActHistory** component to the same GameObject as **Actor**.

Core Classes

Actor

Namespace: **FuzzyBrain** · **Inherits:** **MonoBehaviour** · **Menu:** **FuzzyBrain/Actor**

The evaluation host. Drop one on any GameObject. **Actor** manages the act list, evaluation loop, component caching, idle tracking, and the act lock cycle. If no **FuzzyBrainManager** is present when the Actor enables, one is created automatically with default settings.

Inspector fields

acts (**ScriptableActList**) — The list of Acts to evaluate each tick. **isIdle** (**bool**) — Set by **AddIdleTime()** after **idleDelay** seconds. Cleared by **ResetIdle()**. **isAlive** (**bool**) — General alive flag. Set to **false** by **Die()**. **idleDelay** (**float**) — Seconds before **isIdle** becomes true. Default 1. **_currentAct** (**Act**) — Read-only. Act currently locked. Null when nothing is running.

Properties — **LastFiredAct** (**Act**): most recently fired act, null until first evaluation.
CurrentAct (**Act**): act currently locked, null when idle.

Methods

ActorUpdate() — Runs one full evaluation pass. Called by **FuzzyBrainManager** on its tick schedule. **EnableActor()** — Sorts the act list and resets actor state. Called on **OnEnable**.

Call manually on list swaps. `Refresh()` — Rebuilds component cache and re-sorts the list. Call after adding/removing acts or components at runtime. `SetActList(list)` — Assigns a new act list and calls `Refresh()`. Does not reset actor state. Pass `null` to clear. `ResetActor()` — Resets `isAlive`, idle timer, `isIdle`, and `_currentAct` to initial values. `Die()` — Sets `isAlive = false` and deactivates the GameObject. `ResetIdle()` — Clears `isIdle` and resets the idle timer. Called automatically when `resetIdle = true` on a fired act. `AddIdleTime()` — Accumulates `Time.deltaTime`. Sets `isIdle = true` after `idleDelay` seconds. Call from a fallback act.

Runtime list mutation — for per-actor independence, clone the shared asset and mutate freely:

```
var brain = sharedTemplate.Clone();
actor.SetActList(brain);
brain.Add(newAct);
actor.Refresh();
```

Acts are stateless `ScriptableObject` assets and are safe to share across multiple actors.

Act

Namespace: `FuzzyBrain` · **Inherits:** `ScriptableObject` · **Abstract:** yes

A single rule in an act list. When all conditions pass, `OnStart` and `PerformAct` are called. The actor is optionally locked until `IsComplete` returns true or `maxClockTime` expires. Each distinct behaviour is its own concrete subclass — use the Act Wizard to generate and create instances.

Fields — `conditions` (`Condition[]`): all must pass (AND) for the act to fire. `maxClockTime` (`float`): seconds before `ActClock` forces unlock; `0` = no timeout — always set this when overriding `IsComplete`. `resetIdle` (`bool`): calls `Actor.ResetIdle()` after `PerformAct` returns.

Methods

`CheckConditions(ctx)` — Evaluates all conditions with short-circuit AND. Each Condition SO is evaluated at most once per tick via the condition cache. `OnStart(ctx)` — Called once when this act is first selected. Override for one-shot setup. Default does nothing. `IsComplete(ctx)` — Polled each tick while locked. Return `true` to unlock. Default returns

`true` immediately. `PerformAct(ctx)` — **Abstract**. Implement the behaviour. Use `ctx.Get<T>()` to access components — always null-check.

IsComplete behaviour matrix

Not overridden + `maxClockTime` `0` → fires and unlocks immediately (fire-and-forget). Not

overridden + `maxClockTime` `> 0` → unlocks when `ActClock` exceeds `maxClockTime`.

Overridden + `maxClockTime` `0` → waits for `IsComplete` — no timeout, actor can get stuck.

Overridden + `maxClockTime` `> 0` → waits for `IsComplete`; `ActClock` forces unlock if stuck.

Example — duration-based act:

```
using UnityEngine;
using FuzzyBrain;

[CreateAssetMenu(fileName = "AttackAct", menuName = "FuzzyBrain/Acts/AttackAct")]
public class AttackAct : Act
{
    public override void OnStart(ActContext ctx)
        => ctx.Get<Animator>()?.SetTrigger("Attack");
    public override bool IsComplete(ActContext ctx)
    {
        var anim = ctx.Get<Animator>();
        return anim == null || !anim.GetCurrentAnimatorStateInfo(0).IsName("Attack");
    }
    public override void PerformAct(ActContext ctx) { }
}
```

Notes: place cheap conditions before expensive ones — `CheckConditions` short-circuits. The same Act asset can be shared across multiple lists and actors. Always set `maxClockTime` when overriding `IsComplete`.

Condition and Condition<T>

`Condition` — `FuzzyBrain · ScriptableObject` · Abstract base. Do not subclass directly.

`Condition<T>` — `FuzzyBrain · Condition` · Abstract. `T` : `Component`. The generic base for all user-defined conditions. `T` is fetched from the actor's component cache each tick; the condition is skipped if the actor lacks the component.

Fields — `inverted (bool)`: flips the result of `Verify`. Always apply it manually: `return inverted ? !result : result;`

Members

RequiredType (property) — Returns `typeof(T)`. Sealed in **Condition<T>** — do not override. **Verify(Component)** — Internal dispatch. Sealed in **Condition<T>** — do not override. **Verify(T component)** — **Implement this**. Run the test, apply **inverted**, return the result.

Keep **Verify** fast — it may be called every frame per actor. Never call **GetComponent** inside **Verify**; read only from the passed parameter. Order conditions cheapest-first on each Act — raycasts and physics checks last.

Example:

```
using UnityEngine;
using FuzzyBrain;

[CreateAssetMenu(fileName = "IsLowHealth", menuName =
"FuzzyBrain/Conditions/IsLowHealth")]
public class IsLowHealth : Condition<PlayerStats>
{
    public float threshold = 25f;
    protected override bool Verify(PlayerStats stats)
    {
        bool result = stats.health < threshold;
        return inverted ? !result : result;
    }
}
```

ScriptableActList

Namespace: **FuzzyBrain** · **Inherits:** **ScriptableObject** · **Create:** **FuzzyBrain/ActList**

Holds a prioritised list of Acts. Multiple lists can be authored per actor and swapped at runtime. Field: **list** (**List<Act>**) — the ordered list, sorted automatically on load.

Methods

SortActs() — Sorts **list** descending by condition count. Called on load and **OnValidate**. **MarkDirty()** / **SortIfDirty()** — Mark the list for re-sorting; sort only if dirty. **Add(act)** — Appends an act and marks dirty. Call **Actor.Refresh()** after all mutations. **Remove(act)** — Removes an act and marks dirty. Returns **true** if found. Call **Actor.Refresh()** after.

`Clone()` — New in-memory instance with a shallow copy of this list's acts. Use for per-actor runtime independence.

An act list is a shared asset — mutating `list` directly affects all actors referencing it. Use `Clone()` for per-actor independence. Assign at runtime via `Actor.SetActList(newList)`.

ActContext

Namespace: `FuzzyBrain` · **Type:** `readonly struct`

Zero-allocation per-tick data carrier built once per `Actor.ActorUpdate()` and passed through the entire evaluation chain. Only valid for the duration of one `ActorUpdate()` call — do not store across frames.

Fields — `Actor` (`Actor`), `GameObject` (`GameObject`), `Transform` (`Transform`).

Methods

`Get<T>()` — Returns the cached component of type `T`. Zero-allocation. Returns `null` if absent — always null-check. `Evaluate(condition)` — Evaluates a condition. Results are cached per tick — `Verify()` runs at most once per condition SO per tick. `ForValidation(...)` (*static*) — Creates a validation context for the Validate Act List editor button. `Get<T>()` returns `null` for all types.

FuzzyBrainManager

Namespace: `FuzzyBrain` · **Inherits:** `MonoBehaviour` · **Singleton** · **Menu:** `FuzzyBrain/FuzzyBrainManager`

Drives all registered Actor evaluations in staggered time buckets. Auto-created with default settings if absent when an Actor enables. Add one manually to tune bucket count and tick interval.

Fields — `bucketCount` (`int`, default `4`): number of stagger buckets. `tickInterval` (`float`, default `0.1`): seconds between evaluation passes per actor.

Property — `Instance (FuzzyBrainManager)`: singleton accessor. Null when no manager is in the scene.

Methods — `Register(actor)`: assigns the actor to a bucket; called automatically by `Actor.OnEnable()`. `Unregister(actor)`: removes the actor; called by `Actor.OnDisable()`.

Buckets are offset by `tickInterval / bucketCount`. With 4 buckets and 0.1 s interval, bucket 0 ticks at $t = 0.000, 0.100 \dots$; bucket 1 at $t = 0.025, 0.125 \dots$, and so on. Only one manager should exist per scene — a second instance destroys itself on `Awake`.

Under 50 actors, responsive AI → `bucketCount 2, tickInterval 0.05` 50–200 actors, standard AI → `bucketCount 4, tickInterval 0.1` 200+ actors, background AI → `bucketCount 8, tickInterval 0.2`

ActHistory

Namespace: `FuzzyBrain` · **Inherits:** `MonoBehaviour` · **Opt-in** · **Menu:** `FuzzyBrain/Act History`

Fixed-depth ring buffer of recently completed acts. `Actor` calls `RecordAct` automatically on every unlock — no wiring required. Add to the same `GameObject` as `Actor`.

Fields — `depth (int, default 4)`: acts to retain, minimum 1. `_preview (Act[])`: read-only Inspector preview, most recent first.

Methods — `RecordAct(act)`: writes to buffer position 0; called automatically — do not call manually. `GetAct(offset = 0)`: returns the act at the given offset; 0 = most recent. `Matches(act, offset = 0)`: returns `true` if the act at that offset matches the given asset.

Set `depth` to at least $N - 1$ where N is the longest combo chain. A three-step combo $A \rightarrow B \rightarrow C$ requires `depth ≥ 2`. The backing array is allocated once on `Awake` with no per-frame allocations.

IGizmoDrawable

Namespace: `FuzzyBrain` · **Type:** `interface`

Opt-in interface for conditions that perform spatial queries. Implement on a `Condition<T>` subclass to have `ActorEditor` draw the query volume in the Scene view when the actor is selected.

Method: `DrawGizmo(ActContext ctx)` — draw using `UnityEngine.Gizmos`. Re-run the spatial query inside this method so the gizmo colour reflects the live result. Colour convention: **grey** (edit mode), **green** (play mode, condition true), **red** (play mode, condition false).

```
public void DrawGizmo(ActContext ctx)
{
    Collider2D col = ctx.Get<Collider2D>();
    if (col == null) return;
    float len = col.bounds.extents.y + 0.1f;
    bool hit = Physics2D.Raycast(
        col.transform.position, Vector2.down, len, 1 << groundLayer);
    Gizmos.color = Application.isPlaying
        ? (hit ? Color.green : Color.red) : Color.grey;
    Gizmos.DrawLine(col.transform.position,
        col.transform.position + Vector3.down * len);
}
```

Built-in Conditions

All nine conditions are found under `Assets > Create > FuzzyBrain > Conditions`.

`PreviousActCondition (ActHistory)` — Passes when the act at a history offset matches an expected act asset. `IdleCondition (Actor)` — Passes when `Actor.isIdle` is `true`. `OnGround (Collider2D)` — Downward raycast to a ground layer. Ray = `bounds.extents.y + 0.1f`. `OnSurface (Collider2D)` — Downward raycast excluding the actor's own layer. `AgainstWall (Collider2D)` — Horizontal raycast in a specified direction. Ray = `bounds.extents.x + 0.1f`. `IsTouchingLayer (Collider2D)` — `Physics2D.IsTouchingLayers` contact check — no separate raycast. `FallingCondition (Rigidbody2D)` — Passes when `linearVelocity.y` is below `threshold` (default `-1`). `IsStill (Rigidbody2D)` — Passes when velocity magnitude is below `threshold` (default `1`). `RandomRollCondition (Actor)` — 1-in-N random roll each evaluation. `outOf` (default `5`) = 20% chance.

All conditions expose **inverted** (**bool**): when true, the result of the check is flipped. The four **Collider2D** conditions also implement **IGizmoDrawable** — gizmo colours are grey (edit), green (true), red (false).

PreviousActCondition fields — **expectedAct** (**Act**): the asset to match. **historyOffset** (**int**, default 0): 0 = most recent completed act, 1 = one before that. **OnGround** fields — **groundLayer** (**int**): physics layer index to treat as ground. **OnSurface** fields — **actorLayer** (**int**): the actor's own layer, excluded from the raycast mask. **AgainstWall** fields — **xDirection** (**XDirection**): **Left** or **Right**. **layer** (**int**): physics layer to treat as a wall. **IsTouchingLayer** fields — **layer** (**LayerMask**): layer mask to check contact with. **FallingCondition** / **IsStill** fields — **threshold** (**float**): the velocity boundary value.

Editor Tools

FuzzyBrain Window — **Tools > FuzzyBrain > Open Act Wizard** — **Tools > FuzzyBrain > New Act** or **+ New Act** in the window **Condition Wizard** — **Tools > FuzzyBrain > New Condition** or **Condition Wizard** in the Act detail panel **Settings** — **Edit > Project Settings > FuzzyBrain Create Actor** — Right-click Hierarchy → **FuzzyBrain > Actor**

Create Actor menu item — creates a **GameObject** with **Actor** pre-attached, parents it to the current selection (or scene root if nothing is selected), adds a **FuzzyBrainManager** if one does not exist, and registers full Undo.

Actor Inspector — shows a warning with an **Add FuzzyBrainManager to Scene** button when no manager is in the scene. The **Open FuzzyBrain Editor** button is always shown at the bottom.

FuzzyBrain Window

Main authoring window. Select an Actor in the Hierarchy to activate its list. The window re-evaluates the current selection on focus.

Toolbar — Actor object field (auto-updates on selection), Act list field (drag to switch lists), **+ New List** (prompts for name and folder, creates and assigns a new **ScriptableActList**).

Left pane — Act List

Drag handle  — Reorder rows. Asset saved immediately on drop. `[n]` prefix — Condition count for the Act. `-` / `+` buttons — Remove selected row / append empty slot. **Sort by Conditions**  — Sorts descending by condition count. Matches the runtime sort. **+ New Act** — Opens the Act Wizard. **Validate Act List** — Dry-run validation. Logs warnings for components missing on the Actor.

Right pane — Act Detail — editable Act Name (renames the asset), Max Clock Time, Reset Idle toggle, Conditions list with `inv` toggle and `x` remove per row, and a Condition Wizard button.

Validation — for each act, every condition's `RequiredType` is checked against the Actor's components without entering Play Mode. `PerformAct` is called with a validation context where `ctx.Get<T>()` returns `null`. `NullReferenceException` from missing null-checks is caught and reported as a warning.

Act Wizard

Generate Script tab

Class Name — Valid C# identifier for the new class. Namespace — Optional. Defaults to the value in Project Settings > FuzzyBrain. Scripts Folder — Output directory. Defaults to the configured path. Menu Path — The `CreateAssetMenu` `menuName` written into the generated file.

Create Asset tab — Act Type dropdown (all compiled `Act` subclasses), Asset Name, Assets Folder, Add to Current List toggle (appends to the open list), and a properties preview for configuring conditions before creating.

Condition Wizard

Generate Script tab — Class Name, Namespace, Component Type (the `T` for `Condition<T>`, grouped into Unity and Project components), Scripts Folder, Menu Path.

Create Asset tab — Condition Type dropdown (all compiled `Condition` subclasses), Asset Name, Assets Folder.

Quick Condition tab — generates a complete `Condition<T>` subclass from a field comparison with no `Verify` logic to write. Select Component Type, Left Field (public

fields/properties of type `bool`, `int`, `float`, `double`, or `string`), Operator, and Value or a right-hand Field. A preview HelpBox shows the exact expression. The file opens in the IDE at the `bool result` line. Note: only flat public fields are visible — struct sub-members like `linearVelocity.x` must be written manually.

Settings

Edit > Project Settings > FuzzyBrain — stored in `Assets/FuzzyBrain/Editor/FuzzyBrainSettings.asset`.

Default Namespace — (*empty*) Condition Scripts —

`Assets/FuzzyBrain/Scripts/ActorConditions` Condition Assets —

`Assets/FuzzyBrain/Data/Conditions` Act Scripts —

`Assets/FuzzyBrain/Scripts/Acts` Act Assets — `Assets/FuzzyBrain/Data/Acts` Act

List Assets — `Assets/FuzzyBrain/Data/ActLists`

All folders are created automatically if they do not exist when an asset is saved.

How-To Guides

Writing a Custom Act

1. Generate the script — Open **Tools > FuzzyBrain > New Act > Generate Script**, enter a class name, click **Generate Script**. The file opens in your IDE at the `// TODO` line.

2. Implement `PerformAct` — use `ctx.Get<T>()` to access components. Always null-check the result. Never call `GetComponent` directly and never call `Actor.ResetIdle()` — `Actor` handles `resetIdle` automatically.

3. Override `OnStart` and `IsComplete` (optional) — `OnStart` runs once per lock cycle; use it for triggers and one-shot setup. `IsComplete` is polled each tick while locked; return `true` to unlock. Always set `maxClockTime` on the asset when overriding `IsComplete`.

```

using UnityEngine;
using FuzzyBrain;

[CreateAssetMenu(fileName = "AttackAct", menuName = "FuzzyBrain/Acts/AttackAct")]
public class AttackAct : Act
{
    public override void OnStart(ActContext ctx)
        => ctx.Get<Animator>()?.SetTrigger("Attack");
    public override bool IsComplete(ActContext ctx)
    {
        var anim = ctx.Get<Animator>();
        return anim == null || !anim.GetCurrentAnimatorStateInfo(0).IsName("Attack");
    }
    public override void PerformAct(ActContext ctx) { }
}

```

4. Create an asset instance — After the script compiles, use **Act Wizard > Create Asset** or right-click → **Create > FuzzyBrain/Acts/AttackAct**.

5. Assign conditions and add to a list — Open the **FuzzyBrain Window**, select an Actor, drag the Act into the list, then drag Condition assets into the detail panel.

6. Validate — click **Validate Act List**. The console reports any components missing on the Actor.

Writing a Custom Condition

1. Generate the script — Open **Tools > FuzzyBrain > New Condition > Generate Script**, enter a class name, choose the component the condition reads, click **Generate Script**.

2. Implement **Verify** — read from the passed component parameter, apply **inverted** before returning.

```

using UnityEngine;
using FuzzyBrain;

[CreateAssetMenu(fileName = "IsLowHealth", menuName =
"FuzzyBrain/Conditions/IsLowHealth")]
public class IsLowHealth : Condition<PlayerStats>
{
    public float threshold = 25f;
    protected override bool Verify(PlayerStats stats)
    {
        bool result = stats.health < threshold;
        return inverted ? !result : result;
    }
}

```

Rules: always apply **inverted**. Read only from the passed parameter — never call **GetComponent**. Keep **Verify** fast. Order conditions cheapest-first on each Act — **CheckConditions** short-circuits.

3. Create an asset instance — Condition Wizard > Create Asset or right-click → **Create > FuzzyBrain > Conditions > IsLowHealth**.

4. Assign to an act — drag into the Act's **conditions** array in the Inspector or via the FuzzyBrain Window detail panel.

5. Add gizmos (optional) — implement **IGizmoDrawable** to visualise spatial queries in the Scene view. Re-run the query inside **DrawGizmo** so the colour reflects the live result.

Quick Condition Generator

Open **Tools > FuzzyBrain > New Condition > Quick Condition**. Use it to generate a complete **Condition<T>** from a field comparison when: comparing a numeric field to a constant (**health < 50**), checking a boolean flag (**isGrounded == true**), or comparing two fields on the same component.

Select Component Type → Left Field → Operator → Value (or a right-hand Field) → Class Name → **Generate Script**. The preview HelpBox shows the exact expression before you commit. Note: struct sub-members like **linearVelocity.x** are not visible in the field picker — write those conditions manually.

```
// Generated output for: PlayerStats.health < 50
[CreateAssetMenu(fileName = "IsLowHealth", menuName =
"FuzzyBrain/Conditions/IsLowHealth")]
public class IsLowHealth : Condition<PlayerStats>
{
    protected override bool Verify(PlayerStats component)
    {
        bool result = component.health < 50f;
        return inverted ? !result : result;
    }
}
```

Idle Detection Pattern

`Actor.AddIdleTime()` accumulates `Time.deltaTime` each call. When it exceeds `Actor.idleDelay` (Inspector field, default 1 s), it sets `Actor.isIdle = true`. `Actor.ResetIdle()` clears the flag immediately. No timer component, coroutine, or custom state variable is needed.

Step 1 — Create a zero-condition fallback Act that calls `ctx.Actor.AddIdleTime()` from `PerformAct`. Because it has zero conditions, it fires only when no other act matches.

Step 2 — Create Acts that use `IdleCondition`. These fire once `isIdle` becomes true.

Step 3 — On any Act representing movement or action, set `resetIdle = true`. This calls `ResetIdle()` automatically after `PerformAct` returns.

RunAct	[IsMoving]	resetIdle: true	← resets idle while running
PlayIdleAnimAct	[IdleCondition]	resetIdle: false	← fires once idle threshold met
FallbackAct	(0 conditions)		← accumulates idle time

The sorted order guarantees `FallbackAct` fires last. Set `idleDelay` on the Actor Inspector to control how long inactivity must persist before `isIdle` becomes true.

Act Cooldowns

When an act fires and `IsComplete` returns `false` on the first tick, `Actor` locks to that act — no other acts are evaluated until unlock. On every subsequent tick, `IsComplete(ctx)` is polled. If it never returns `true`, `maxClockTime` forces an unlock. The lock state lives on `Actor` and `ActClock`, not on the Act asset — multiple actors sharing the same Act each have independent lock state.

Fire-and-forget — override only `PerformAct`. `IsComplete` returns `true` immediately; no lock is set.

Fixed-duration lock — leave `IsComplete` at default; set `maxClockTime` on the asset. The `ActClock` drives the unlock.

AttackAct maxClockTime: 0.75 ← locked for 0.75 s, then resumes evaluation

Completion-signal lock — override `IsComplete` and always set `maxClockTime` as a safety timeout.

```
public override bool IsComplete(ActContext ctx)
{
    var anim = ctx.Get<Animator>();
    return anim == null || !anim.GetCurrentAnimatorStateInfo(0).IsName("Attack");
}
// Set maxClockTime: 2.0 on the asset - forces unlock if animator gets stuck
```

Choosing `maxClockTime` — for animation-driven acts use `animationLength + 0.15f` as a buffer. For physics-driven acts use `0.5–2.0f`. `0` means no timeout — only use this if `IsComplete` is guaranteed to return `true`. Values shorter than `FuzzyBrainManager.tickInterval` have no practical effect.

Combo Sequences

Combos require `ActHistory` on the actor and `PreviousActCondition` assets. No custom code is needed for basic combos.

When an act unlocks, `Actor` calls `ActHistory.RecordAct` automatically.

`PreviousActCondition` with `historyOffset = 0` checks the most recently completed act. Chain multiple `PreviousActCondition` assets on one act to require an exact sequence.

The combo window equals `maxClockTime` of the preceding act.

Setup:

1. Add **Act History** to the Actor GameObject. Set **depth** to at least **N - 1** where N is the longest combo chain (two-step = **depth 1**, three-step = **depth 2**).
2. Each act in the chain should override **OnStart** and **IsComplete** to stay locked for a meaningful input window.
3. Create **PreviousActCondition** assets: **expectedAct** = the preceding act, **historyOffset** = its position in history.
4. Assign conditions. The act list sorts by condition count — the finisher (most conditions) is always evaluated first automatically.

```
FinisherAttack [IsPressingFinisher, PrevAct=Heavy@0, PrevAct=Light@1] maxClockTime:
1.2
HeavyAttack    [IsPressingHeavy, PrevAct=Light@0] maxClockTime:
0.8
LightAttack    [IsPressingLight] maxClockTime:
0.5
Idle           []
```

To widen the combo window: increase **maxClockTime** on the preceding act. To narrow it: decrease it. If the player misses the window, the unlock occurs and history conditions no longer match.

Reading history from code:

```
var history = ctx.Get<ActHistory>();
if (history == null) return;
bool wasLight = history.Matches(lightAttackAsset, 0);
```

Notes: fire-and-forget acts record to history on the same tick they fire. **ActHistory** records all unlocking acts including the idle fallback.

Swapping Act Lists at Runtime

```
using UnityEngine;
using FuzzyBrain;

public class BehaviourSwitcher : MonoBehaviour
{
    [SerializeField] private Actor      _actor;
    [SerializeField] private ScriptableActList _combatList;
    [SerializeField] private ScriptableActList _idleList;

    /// <summary>Switches the actor to its combat behaviour profile.</summary>
    public void EnterCombat()
    {
        _actor.acts = _combatList;
        _actor.EnableActor(); // re-sorts list and resets actor state
    }

    /// <summary>Switches the actor to its idle behaviour profile.</summary>
    public void LeaveCombat()
    {
        _actor.acts = _idleList;
        _actor.EnableActor();
    }
}
```

`EnableActor()` sorts the new list by condition count and resets `isIdle` and `isAlive`. Always call it after swapping. Act lists are shared assets — use `Instantiate(myList)` for per-actor independence. The component cache built on `Awake` does not change when swapping; conditions in the new list must reference components already present on the actor.

Using FuzzyBrainManager

Without a manager, Actors self-tick every `Update()`. A manager changes only the timing — behaviour is otherwise identical. Add one manually to spread evaluation cost across frames.

Setup — create an empty GameObject, add a **FuzzyBrainManager** component (or use the **Add FuzzyBrainManager to Scene** button in any Actor Inspector warning), then set **Bucket Count** and **Tick Interval**. All Actors in the scene, and any instantiated later, register automatically via `Actor.OnEnable`.

Set `bucketCount` and `tickInterval` based on actor count — see the Recommended Values in the `FuzzyBrainManager` class reference in Part 1. Manual registration: `FuzzyBrainManager.Instance?.Register(myActor) / Unregister(myActor)`.

Tutorial — Wall Bouncer

A character that walks back and forth between two walls, built end-to-end with FuzzyBrain.

Scene and Actor Setup

Create a scene with a ground plane, a left wall, a right wall, and a character sprite with `Rigidbody2D` and `BoxCollider2D`. Select the character, add the **Actor** component, and click **Add FuzzyBrainManager to Scene** if prompted. Open **Tools > FuzzyBrain > Open**, select the character, click **+ New List**, name it "WallBouncer", and choose a save folder.

Movement Component

Acts delegate movement to a dedicated component rather than moving the character directly. Add `WalkerBehaviour`:

```
using UnityEngine;

public class WalkerBehaviour : MonoBehaviour
{
    public float speed = 3f;
    private Rigidbody2D _rb;

    private void Awake() => _rb = GetComponent<Rigidbody2D>();

    /// <summary>Applies a leftward horizontal velocity.</summary>
    public void MoveLeft()
        => _rb.linearVelocity = new Vector2(-speed, _rb.linearVelocity.y);

    /// <summary>Applies a rightward horizontal velocity.</summary>
    public void MoveRight()
        => _rb.linearVelocity = new Vector2(speed, _rb.linearVelocity.y);
}
```

Acts

Use **Act Wizard > Generate Script** to create `MoveLeftAct` and `MoveRightAct`. Implement both:

```
using UnityEngine;
using FuzzyBrain;

[CreateAssetMenu(fileName = "MoveLeftAct", menuName = "FuzzyBrain/Acts/MoveLeftAct")]
public class MoveLeftAct : Act
{
    public override void PerformAct(ActContext ctx)
    {
        var walker = ctx.Get<WalkerBehaviour>();
        if (walker == null) return;
        walker.MoveLeft();
    }
}
```

`MoveRightAct` is identical but calls `walker.MoveRight()`. After compiling, use **Act Wizard > Create Asset** to create asset instances of both.

Conditions

Navigate to **Assets > Create > FuzzyBrain > Conditions** and create two **AgainstWall** assets:

AgainstRightWall — `xDirection: Right`, `layer: your wall layer`. **AgainstLeftWall** — `xDirection: Left`, `layer: your wall layer`.

Wiring the Act List

`MoveLeftAct` conditions: [AgainstRightWall] ← fires when against the right wall
`MoveRightAct` conditions: (none) ← fallback, always fires otherwise

`MoveLeftAct` has one condition so it is checked first. While the character is not against the right wall it falls through to `MoveRightAct`. Drag **AgainstRightWall** into `MoveLeftAct`'s conditions. Click **Sort by Conditions** ↓ to confirm order, then click **Validate Act List** to check for missing components. Enter Play Mode — the character walks right, hits the wall, and walks left.

Writing a Custom Condition (Velocity-Based Alternative)

Instead of a physics ray, check horizontal velocity direction with a custom condition:

```
using UnityEngine;
using FuzzyBrain;

[CreateAssetMenu(fileName = "MovingRight", menuName =
"FuzzyBrain/Conditions/MovingRight")]
public class MovingRight : Condition<Rigidbody2D>
{
    protected override bool Verify(Rigidbody2D rb)
    {
        bool result = rb.linearVelocity.x > 0f;
        return inverted ? !result : result;
    }
}
```

The generic parameter **Rigidbody2D** tells FuzzyBrain which component to pull from the cache. Always apply **inverted** before returning — without it the Inspector's invert toggle silently does nothing.

For simple field comparisons, use the **Quick Condition** tab in the Condition Wizard instead. It generates the **Verify** body automatically. It only sees flat public fields — struct sub-members like **linearVelocity.x** must be written by hand.